

Qingluan Post-Quantum Digital Signature Algorithm Text

Wu Guangfu, Liu Lei, Wu Rui, Liu Lulu, Li Kangjun

College of Software Engineering, Jiangxi University of Science and Technology, China

Abstract

This project delivers Qingluan, a complete post-quantum digital signature solution built on coding theory, with the Restricted Syndrome Decoding Problem (R-SDP) over a multiplicative restricted subgroup as its security foundation. The scheme follows the CROSS-RSDP protocol, combining an MPC-in-the-Head zero-knowledge identification protocol with the Fiat-Shamir transform to obtain non-interactive signatures, and provides four deployable security levels: Qingluan-128, Qingluan-256, Qingluan-384, and Qingluan-512. Our main technical achievement is an end-to-end architecture built entirely on national-cryptography primitives (SM3) with strict domain separation. The prover proves knowledge of a restricted error e with $eH^\top = s$ through t fixed-weight parallel rounds, and all commitments and the message digest are salted (eTCR) so that security reduces to (second-)preimage / target-collision resistance rather than plain collision resistance. The project also produced a full parameter and engineering specification, including size formulas, complexity analysis, and ROM/QROM EUF-CMA security statements with constant-time and side-channel-oriented protections. On implementation, we delivered a C11 reference version and GM/T 0028-aligned self-test interfaces, and validated portability and ecosystem compatibility. Performance evaluation on x86-64 Windows confirms practical deployability (signing in milliseconds to under one second), while identifying large signature size as the primary tradeoff and outlining clear optimization paths.

Contents

1	Algorithm Description	3
1.1	Overview	3
1.2	Symbol Definitions	3
1.3	Decoding Issues of the Restricted Syndrome	3
1.4	Parity-Check Matrix Structure	4
1.5	Cryptographic Primitives	4
1.6	Domain Separation	5
1.7	Key Generation Algorithm	6
1.8	Signature Algorithm	6
1.9	Verification Algorithm	8
1.10	Round-Seed Derivation and Path	9
1.11	Parameter Set	9
1.12	Data Size	9
2	Design Principles	10
2.1	The Mathematical Hard Problem on Which the Algorithm Is Based	10
2.2	Basis for Parameter Selection	10
2.3	Failure Probability Analysis	11
2.4	Other Considerations	12
3	Safety Statement and Analysis	13
3.1	Theoretical Safety	13
3.2	Concrete Safety Measures	14
3.3	Other Safety Features	16
4	Performance Evaluation	18
4.1	Performance Analysis	19
4.2	Performance Testing	20
5	Feature Statement	22
5.1	Innovation of the Algorithm	22
5.2	Simplicity of the Algorithm	23
5.3	Flexibility of the Algorithm	23
5.4	Algorithm Compatibility	23
5.5	Scalability of the Algorithm	24
5.6	Cross-Platform Implementation Performance Advantages	24

1 Algorithm Description

1.1 Overview

Qingluan is a post-quantum digital signature scheme based on coding theory. Its security rests on the Restricted Syndrome Decoding Problem (R-SDP) over a restricted multiplicative subgroup $E = \langle g \rangle$ of the prime field $\mathbb{F}_p$: given $H \in \mathbb{F}_p^{(n-k) \times n}$ and s , find $e \in E^n$ with $eH^\top = s$. Qingluan instantiates the CROSS-RSDP protocol — an MPC-in-the-Head 5-pass Σ -protocol made non-interactive by the Fiat–Shamir transform — and proves knowledge of e through t fixed-weight parallel rounds.

Qingluan employs the national standard SM3 hash algorithm (GB/T 32905-2016) as its underlying hash primitive, and builds multi-pipeline hashing and scalable output functions (XOF) based on SM3. Random number generation utilizes the SM3-based Hash-DRBG (GM/T 0005).

Qingluan offers four parameter sets — Qingluan-128, Qingluan-256, Qingluan-384, and Qingluan-512 — corresponding to classical security strengths of 128, 256, 384, and 512 bits, respectively. All levels share the same fixed base algebra ($p = 127$, $z = 7$, $g = 2$) and differ only in the code/round parameters (n, k, t, w) .

1.2 Symbol Definitions

symbol	meaning
p	Order of the prime base field $\mathbb{F}_p$ (fixed at 127 for every level)
g	Generator of the restricted subgroup (fixed at 2)
z	Order of the restricted subgroup, $ E = z$ (fixed at 7)
E	Restricted multiplicative subgroup $E = \langle g \rangle = \{1, 2, 4, 8, 16, 32, 64\} \subset \mathbb{F}_p^*$
n	Code length
k	Code dimension
$r = n - k$	Redundancy (number of rows of H)
H	Parity-check matrix $H = [V \mid I_{n-k}] \in \mathbb{F}_p^{(n-k) \times n}$ (identity on the right)
V	Uniform-random left block of H , an $(n - k) \times k$ matrix over $\mathbb{F}_p$
s	Syndrome $s = eH^\top \in \mathbb{F}_p^{n-k}$ (part of the public key)
e	Restricted error vector, $e \in E^n$ (every coordinate lies in E)
η	Exponent vector $\eta \in \mathbb{F}_z^n$ with $e_j = g^{\eta_j}$
t	Number of parallel rounds (Fiat–Shamir repetitions)
w	Weight of the fixed-weight second challenge $\text{chall2} \in B(t, w)$
c	Domain constant $c = 2t - 1$
Seed_sk	2λ -bit secret-key seed (the entire sk)
Seed_pk	2λ -bit public seed from which H is derived
Salt	2λ -bit per-signature salt (the eTCR randomizer)
λ	Security parameter (bits)
SM3	National hash algorithm (GB/T 32905-2016), 256-bit output
Hash	Multi-pipe SM3, output widened to 2λ bits (PARAM_HASH_BYTES)
XOF	SM3 counter-mode extendable-output function (the CSPRNG)
DRBG	SM3-based Hash-DRBG (GM/T 0005)
pack/unpack	Little-endian bit-packing of $\mathbb{F}_p$ (7-bit) / $\mathbb{F}_z$ (3-bit) vectors, and its inverse

1.3 Decoding Issues of the Restricted Syndrome

Definition (R-SDP). Let p be prime and $E = \langle g \rangle$ the order- z multiplicative subgroup of $\mathbb{F}_p^*$. Given a matrix $H \in \mathbb{F}_p^{(n-k) \times n}$ and a syndrome $s \in \mathbb{F}_p^{n-k}$, find a restricted error vector $e \in E^n$ such that

$H \cdot e^\top = s$, where every coordinate satisfies

$$e_i \in E \iff e_i^z = 1 \iff e_i = g^{\eta_i} \text{ for some exponent } \eta_i \in \mathbb{F}_z = \{0, \dots, z-1\}.$$

There is no Hamming-weight constraint: all n coordinates are non-zero and lie in the subgroup E (this is what makes the problem *restricted*).

R-SDP is a restricted variant of the classical Syndrome Decoding Problem (SDP, NP-complete): instead of bounding the Hamming weight, it confines every coordinate to the multiplicative subgroup E . This added algebraic structure both keeps the problem hard and enables an efficient MPC-in-the-Head membership check. CROSS-RSDP fixes $p = 127$, $z = 7$, $g = 2$.

1.4 Parity-Check Matrix Structure

Qingluan uses a uniform-random parity-check matrix in systematic form (no quasi-cyclic / block-circulant structure):

$$H = [V \mid I_{n-k}].$$

Here I_{n-k} is the $(n-k) \times (n-k)$ identity placed on the *right*, and V is a uniform-random $(n-k) \times k$ matrix over $\mathbb{F}_p$. Writing $e = (e_A \parallel e_B)$ with $e_A \in \mathbb{F}_p^k$ and $e_B \in \mathbb{F}_p^{n-k}$, the syndrome is $s = V \cdot e_A^\top + e_B$.

Matrix expansion process.

- Use the XOF as CSPRNG $\mathbb{F}_p$ with input (`DOMAIN_MATRIX` $\parallel$ `Seed_pk`) to generate the random field elements of V .
- Sample all $(n-k) \cdot k$ entries of V directly from the XOF stream; there are no circulant blocks and no first-row replication.
- The identity block I_{n-k} is implicit and is never sampled or stored.
- Field elements are produced by rejection sampling: read 7 bits from the XOF (mask `0x7F`); accept if the value is < 127 , otherwise retry (acceptance $127/128 > 99\%$).

This structure ensures that:

- The public key is seed-compressed: only `Seed_pk` and `pack(s)` are stored, so $|\text{pk}|$ stays tiny (77–302 B).
- With $H = [V \mid I_{n-k}]$, the syndrome simplifies to $s = V \cdot e_A^\top + e_B$.

1.5 Cryptographic Primitives

1.5.1 SM3 Hash Algorithm

It uses the national standard SM3 (GB/T 32905-2016) as the underlying hash primitive, producing a fixed 256-bit (32-byte) digest.

1.5.2 Multi-Pipe Hash

To widen the SM3 output to 2λ bits, Qingluan concatenates `HASH_PIPES` domain-separated SM3 pipes. Honest claim: this widens preimage / second-preimage / target-collision (TCR) resistance to the full 2λ bits; it does *not* raise plain collision resistance beyond $\sim 2^{128}$ (Joux multicollision), so every collision-relying use is salted (eTCR).

Number of pipes:

$$\text{HASH_PIPES} = \lceil \text{PARAM_HASH_BYTES} / 32 \rceil.$$

- Qingluan-128: `HASH_PIPES = 1` (standard SM3, no prefix; 256-bit output).
- Qingluan-256: `HASH_PIPES = 2` (512-bit output).
- Qingluan-384: `HASH_PIPES = 3` (768-bit output); Qingluan-512: `HASH_PIPES = 4` (1024-bit output).

Construction method. For `HASH_PIPES > 1`, pipeline i computes $\text{SM3}(\text{byte}(i) \parallel \text{message})$:

- All pipelines use the same message but different prefix bytes to implement domain separation.
- $\text{Output} = \text{pipe}_0\text{-digest} \parallel \text{pipe}_1\text{-digest} \parallel \dots \parallel \text{pipe}_{P-1}\text{-digest}$.
- The output of the last pipeline can be truncated to the total length of `PARAM_HASH_BYTES`.

Security. Preimage / second-preimage / TCR resistance = 2λ bits classical (λ bits quantum); plain collision resistance remains $\sim 2^{128}$ regardless of width (Joux), which is why no security claim depends on it. For `HASH_PIPES = 1` (Qingluan-128), no prefix bytes are added, and the standard SM3 behavior is maintained.

1.5.3 Scalable Output Function

The XOF process consists of two stages: absorption and squeezing.

Absorption phase. Process inputs of any length using the incremental interface of multi-pipe Hash: initialize the multi-pipe hash context; enter data sequentially (`hash_update`); finalize (`hash_final`) to generate a key of `PARAM_HASH_BYTES` bytes.

Squeezing phase. Generate output of arbitrary length using the SM3 counter mode:

$$O_i = \text{SM3}(\text{key} \parallel \text{BE}(i)), \quad i = 0, 1, 2, \dots$$

Each block provides 32 bytes of pseudo-random output, and incremental squeezing is supported.

1.5.4 Random Number Generator

Use an SM3-based Hash-DRBG (GM/T 0005):

- Seed length: 32 bytes.
- Entropy source: operating-system CSPRNG (Windows: `BCryptGenRandom`; Linux: `/dev/urandom`).
- Re-seeding interval: auto-reseed after every 1024 outputs.
- State: thread-local storage, prevents multithreaded contention.

Procedure: if not initialized, obtain 64 bytes of seed material from the system CSPRNG and initialize the internal state V and constant C ; if the reseed count exceeds the threshold, obtain new entropy from the system CSPRNG and update the internal state; output generation computes $h = \text{SM3}(0x02 \parallel V)$, extracts the required bytes, and updates V and the counter.

1.6 Domain Separation

All hash and XOF calls use domain-separation tags as input prefixes to prevent collisions between hash values for different purposes:

label	value	use
<code>DOMAIN_EXPAND_SK</code>	<code>0x00</code>	Derive (<code>Seed_e</code> , <code>Seed_pk</code>) from <code>Seed_sk</code>

label	value	use
DOMAIN_MATRIX	0x01	Expand V , hence $H = [V \mid I_{n-k}]$
DOMAIN_ERROR	0x02	Sample the secret exponents η ($e = g^\eta$)
DOMAIN_SEEDLEAVES	0x03	Derive the t per-round seeds $\text{Seed}[i]$
DOMAIN_ROUND	0x04	Per-round expansion of $(\eta'[i], u'[i])$
DOMAIN_CMT0	0x05	$\text{cmt0}[i] = \text{Hash}(s'[i] \parallel v[i] \parallel \text{Salt} \parallel i+c)$
DOMAIN_CMT1	0x06	$\text{cmt1}[i] = \text{Hash}(\text{Seed}[i] \parallel \text{Salt} \parallel i+c)$
DOMAIN_DIGEST_CMT0	0x07	$\text{digest_cmt0} = \text{Hash}(\text{cmt0}[1..t])$
DOMAIN_DIGEST_CMT1	0x08	$\text{digest_cmt1} = \text{Hash}(\text{cmt1}[1..t])$
DOMAIN_DIGEST_CMT	0x09	$\text{digest_cmt} = \text{Hash}(\text{digest_cmt0} \parallel \text{digest_cmt1})$
DOMAIN_MSG	0x0A	Salted message digest $\text{Hash}(\text{Salt} \parallel \text{pk_hash} \parallel \text{Msg})$
DOMAIN_CHALL1	0x0B	$\text{digest_chall1} = \text{Hash}(\text{digest_Msg} \parallel \text{digest_cmt} \parallel \text{Salt})$
DOMAIN_CHALL1_GEN	0x0C	Expand $\text{chall1} \in (\mathbb{F}_p^*)^t$
DOMAIN_CHALL2	0x0D	$\text{digest_chall2} = \text{Hash}(y[1..t] \parallel \text{digest_chall1})$
DOMAIN_CHALL2_GEN	0x0E	Expand $\text{chall2} \in B(t, w)$
DOMAIN_COMMIT	0x0F	Generic salted TCR commitment

1.7 Key Generation Algorithm

Input: security parameter λ .

Output: public key $\text{pk} = (\text{Seed_pk}, \text{pack}(s))$; private key $\text{sk} = \text{Seed_sk}$ (a single 2λ -bit seed).

1. Sample the secret-key seed: $\text{Seed_sk} \leftarrow \text{DRBG}(2\lambda/8 \text{ bytes})$.
2. Derive $(\text{Seed_e}, \text{Seed_pk}) \leftarrow \text{CSPRNG}(\text{DOMAIN_EXPAND_SK} \parallel \text{Seed_sk})$, each $2\lambda/8$ bytes.
3. Expand $V \leftarrow \text{CSPRNG}_{\mathbb{F}_p^{(n-k) \times k}}(\text{DOMAIN_MATRIX} \parallel \text{Seed_pk})$ and set $H = [V \mid I_{n-k}]$.
4. Sample $\eta \leftarrow \text{CSPRNG}_{\mathbb{F}_z^n}(\text{DOMAIN_ERROR} \parallel \text{Seed_e})$ and lift to the restricted error $e_j = g^{\eta_j} \in E^n$.
5. Compute the syndrome $s = eH^\top = V \cdot e_A^\top + e_B \in \mathbb{F}_p^{n-k}$.
6. Serialize the public key: $\text{pk} = \text{Seed_pk} \parallel \text{pack}(s)$.
7. Store the private key: $\text{sk} = \text{Seed_sk}$ (the single 2λ -bit seed; everything else is re-derived).
8. Securely erase all intermediate sensitive data (η, e, V, s) .

Restricted error generation ($e \in E^n$, full length, constant-time):

- For each coordinate $j = 0, \dots, n-1$: sample an exponent $\eta_j \in \mathbb{F}_z = \{0, \dots, z-1\}$ from the XOF by rejection (3-bit mask, accept if < 7).
- Map to the restricted group: $e_j = g^{\eta_j}$ via a constant-time table lookup over $E = \{1, 2, 4, 8, 16, 32, 64\}$.
- Every coordinate is non-zero and lies in E ; there is no position-shuffling and no Hamming-weight selection (unlike classical SDP).

Public key size: $2\lambda/8$ (Seed_pk) + $\lceil r \cdot 7/8 \rceil$ ($\text{pack}(s)$) bytes = 77/153/227/302 B. Private key size: $2\lambda/8$ bytes (a single Seed_sk) = 32/64/96/128 B.

1.8 Signature Algorithm

Input: private key $\text{sk} = \text{Seed_sk}$, message $m \in \{0, 1\}^*$.

Output: signature σ .

Phase 1.

- $(\eta, H) \leftarrow \text{ExpandSK}(\text{Seed_sk})$: re-derive $(\text{Seed_e}, \text{Seed_pk}) = \text{CSPRNG}(\text{Seed_sk})$, expand V , set $H = [V \mid I_{n-k}]$, and recover the secret exponents $\eta \in \mathbb{F}_z^n$.
- Lift the secret error $e = g^\eta \in E^n$.
- Recompute the syndrome $s = eH^\top = V \cdot e_A^\top + e_B$ and the public-key bytes $\text{pk} = \text{Seed_pk} \parallel \text{pack}(s)$.
- Compute the public-key hash $\text{pk_hash} = \text{Hash}(\text{Seed_pk} \parallel \text{pack}(s))$ for BUFF-style key binding.
- Sample a fresh per-signature salt $\text{Salt} \leftarrow \{0, 1\}^{2\lambda}$ and a root seed $\text{Seed} \leftarrow \{0, 1\}^\lambda$.
- Compute the salted message digest $\text{digest_Msg} = \text{Hash}(\text{DOMAIN_MSG} \parallel \text{Salt} \parallel \text{pk_hash} \parallel m)$; the Salt is the eTCR randomizer.
- Derive the t per-round seeds $\text{Seed}[1..t] \leftarrow \text{SeedLeaves}(\text{DOMAIN_SEEDLEAVES} \parallel \text{Seed} \parallel \text{Salt})$ as t *independent* per-index expansions (fast-style; no GGM tree).

Phase 2. For each round $i = 1, \dots, t$:

- $(\eta'[i], u'[i]) \leftarrow \text{CSPRNG}(\text{DOMAIN_ROUND} \parallel \text{Seed}[i] \parallel \text{Salt} \parallel i+c)$, with $\eta'[i] \in \mathbb{F}_z^n$ and $u'[i] \in \mathbb{F}_p^n$.
- $v[i] = \eta - \eta'[i] \pmod{z}$; lift $v_E[i] = g^{v[i]}$ and $e'_E[i] = g^{\eta'[i]}$; form $u[i] = v_E[i] \star u'[i]$ and $s'[i] = u[i]H^\top$.
- $\text{cmt0}[i] = \text{Hash}(\text{DOMAIN_CMT0} \parallel s'[i] \parallel v[i] \parallel \text{Salt} \parallel i+c)$ — this binds the secret-dependent $v[i]$; $\text{cmt1}[i] = \text{Hash}(\text{DOMAIN_CMT1} \parallel \text{Seed}[i] \parallel \text{Salt} \parallel i+c)$.
- $\text{digest_cmt0} = \text{Hash}(\text{cmt0}[1..t])$; $\text{digest_cmt1} = \text{Hash}(\text{cmt1}[1..t])$; $\text{digest_cmt} = \text{Hash}(\text{digest_cmt0} \parallel \text{digest_cmt1})$.

Phase 3.

- $\text{digest_chall1} = \text{Hash}(\text{DOMAIN_CHALL1} \parallel \text{digest_Msg} \parallel \text{digest_cmt} \parallel \text{Salt})$.
- $\text{chall1} \leftarrow \text{CSPRNG} - (\mathbb{F}_p^*)^t(\text{DOMAIN_CHALL1_GEN} \parallel \text{digest_chall1} \parallel t+c)$: one non-zero scalar per round.

Phase 4. For each round $i = 1, \dots, t$:

- $y[i] = u'[i] + \text{chall1}[i] \cdot e'_E[i] \in \mathbb{F}_p^n$.
- $\text{digest_chall2} = \text{Hash}(\text{DOMAIN_CHALL2} \parallel y[1..t] \parallel \text{digest_chall1})$.
- $\text{chall2} \leftarrow \text{CSPRNG} - B(t, w)(\text{DOMAIN_CHALL2_GEN} \parallel \text{digest_chall2} \parallel t+c+1)$: a fixed-weight binary string of weight w . Convention: $\text{chall2}[i] = 1$ reveals $\text{Seed}[i]$ (cmt1 branch, proves e' restricted); $\text{chall2}[i] = 0$ reveals $(y[i], v[i], \text{cmt1}[i])$ (cmt0 branch, proves the syndrome relation). Exactly w rounds are type-1.
- $\text{Path} = \{\text{Seed}[i] : \text{chall2}[i] = 1\}$ (w seeds); $\text{Proof} = \{\text{cmt0}[i] : \text{chall2}[i] = 1\}$ (w commitments).
- $\text{resp} =$ for each round with $\text{chall2}[i] = 0$: $(\text{pack}(y[i]), \text{pack}(v[i]), \text{cmt1}[i])$.

Phase 5. Byte format of the signature σ :

```

 $\sigma = \text{Salt (PARAM\_SALT\_BYTES)}$ 
 $\parallel \text{digest\_cmt (PARAM\_HASH\_BYTES)}$ 
 $\parallel \text{digest\_chall2 (PARAM\_HASH\_BYTES)}$ 
 $\parallel \text{Path: } w \times \text{Seed}[i] \text{ (each PARAM\_SEED\_BYTES; chall2}[i] = 1 \text{ rounds)}$ 
 $\parallel \text{Proof: } w \times \text{cmt0}[i] \text{ (each PARAM\_HASH\_BYTES; chall2}[i] = 1 \text{ rounds)}$ 
 $\parallel \text{resp: for each of the } (t - w) \text{ rounds with chall2}[i] = 0 :$ 
 $\quad \text{pack}(y[i]) \text{ (QINGLUAN\_Y\_BYTES)} \parallel \text{pack}(v[i]) \text{ (QINGLUAN\_V\_BYTES)}$ 
 $\quad \parallel \text{cmt1}[i] \text{ (PARAM\_HASH\_BYTES)}.$ 

```

Finally, securely delete all intermediate sensitive data.

1.9 Verification Algorithm

Input: public key $\text{pk} = (\text{Seed_pk}, \text{pack}(s))$, message m , signature σ .

Output: Accept / Reject.

Phase 1: Parsing and Initialization.

- Parse pk : Seed_pk and $s = \text{unpack}(s_{\text{packed}})$.
- Expand $V \leftarrow \text{CSPRNG}_{-\mathbb{F}_p^{(n-k) \times k}}(\text{DOMAIN_MATRIX} \parallel \text{Seed_pk})$ and set $H = [V \mid I_{n-k}]$.
- Parse σ into $\text{Salt}, \text{digest_cmt}, \text{digest_chall2}, \text{Path}, \text{Proof}, \text{resp}$; reject immediately if any length or range is malformed.
- Compute $\text{pk_hash} = \text{Hash}(\text{Seed_pk} \parallel \text{pack}(s))$.
- Compute $\text{digest_Msg} = \text{Hash}(\text{DOMAIN_MSG} \parallel \text{Salt} \parallel \text{pk_hash} \parallel m)$.
- Recompute $\text{digest_chall1} = \text{Hash}(\text{DOMAIN_CHALL1} \parallel \text{digest_Msg} \parallel \text{digest_cmt} \parallel \text{Salt})$; then $\text{chall1} \leftarrow \text{CSPRNG}_{-(\mathbb{F}_p^*)^t}$ and $\text{chall2} \leftarrow \text{CSPRNG}_{-B(t, w)}$ from digest_chall2 .

Phase 2: Reconstruction. Rebuild the w revealed seeds $\{\text{Seed}[i] : \text{chall2}[i] = 1\}$ from Path .

For each round $i = 1, \dots, t$:

- If $\text{chall2}[i] = 1$ (cmt1 branch): $\text{cmt1}[i] = \text{Hash}(\text{DOMAIN_CMT1} \parallel \text{Seed}[i] \parallel \text{Salt} \parallel i+c)$; reconstruct $(\eta'[i], u'[i])$ from $\text{Seed}[i]$; set $e'_E[i] = g^{\eta'[i]}$ and $y[i] = u'[i] + \text{chall1}[i] \cdot e'_E[i]$; take $\text{cmt0}[i]$ from Proof .
- If $\text{chall2}[i] = 0$ (cmt0 branch): read $(y[i], v[i], \text{cmt1}[i])$ from resp ; CHECK $v[i] \in \mathbb{F}_z^n$ (every exponent $< z$) and reject otherwise — this restricted-ness check is exactly what v_1 lacked. Then $v_E[i] = g^{v[i]}$, $y'[i] = v_E[i] \star y[i]$, $s'[i] = y'[i]H^\top - \text{chall1}[i] \cdot s$, and $\text{cmt0}[i] = \text{Hash}(\text{DOMAIN_CMT0} \parallel s'[i] \parallel v[i] \parallel \text{Salt} \parallel i+c)$.

Recompute $\text{digest_cmt0} = \text{Hash}(\text{cmt0}[1..t])$ from the recomputed and Proof -supplied commitments; $\text{digest_cmt1} = \text{Hash}(\text{cmt1}[1..t])$ and $\text{digest_cmt}' = \text{Hash}(\text{digest_cmt0} \parallel \text{digest_cmt1})$; and $\text{digest_chall2}' = \text{Hash}(\text{DOMAIN_CHALL2} \parallel y[1..t] \parallel \text{digest_chall1})$.

Phase 3: Consistency Verification.

- Check $\text{digest_cmt} \stackrel{?}{=} \text{digest_cmt}'$ (constant-time comparison).
- Check $\text{digest_chall2} \stackrel{?}{=} \text{digest_chall2}'$ (constant-time comparison).
- If all validations pass, return “Accept”; otherwise, return “Reject”.

1.10 Round-Seed Derivation and Path

Qinglun follows the fast-style direct form: the t per-round seeds are derived *independently* from the root seed, so there is no GGM / binary seed tree and no N -party MPC. Each round seed is

$$\text{Seed}[i] = \text{CSPRNG}(\text{DOMAIN_SEEDLEAVES} \parallel \text{Seed} \parallel \text{Salt} \parallel i), \quad i = 1, \dots, t.$$

- Because the seeds are independent, revealing the w seeds with $\text{chall2}[i] = 1$ leaks nothing about the $t - w$ unrevealed seeds.
- *Path* (revealed seeds): the signature carries exactly the w seeds $\{\text{Seed}[i] : \text{chall2}[i] = 1\}$, which let the verifier recompute the cmt1 branch for those rounds.
- *Proof* (revealed commitments): the signature also carries the w commitments $\{\text{cmt0}[i] : \text{chall2}[i] = 1\}$; for the remaining $t - w$ rounds, $\text{cmt0}[i]$ is recomputed by the verifier.
- Compression note: replacing the direct Path/Proof with a GGM seed tree or a Merkle tree would shrink the signature when $w \approx t$, but changes only the signature SIZE, never the soundness; it is Part-2 scope.

parameter	Qinglun-128	Qinglun-256	Qinglun-384	Qinglun-512
p (order of base field $\mathbb{F}_p$)	127	127	127	127
z (order of restricted subgroup $ E $)	7	7	7	7
g (generator of $E = \langle g \rangle$)	2	2	2	2
n (code length)	127	251	370	491
k (code dimension)	76	150	221	293
$r = n - k$ (redundancy)	51	101	149	198
t (number of parallel rounds)	256	512	763	1018
w (fixed weight of 2nd challenge)	212	424	631	842
λ (security parameter / bit)	128	256	384	512
PARAM_SEED_BYTES ($\lambda/8$)	16	32	48	64
PARAM_SALT_BYTES ($2\lambda/8$)	32	64	96	128
PARAM_HASH_BYTES ($2\lambda/8$)	32	64	96	128
HASH_PIPES ($\lceil 2\lambda/256 \rceil$)	1	2	3	4
fq_t type ($p = 127 \leq 255$)	uint8_t	uint8_t	uint8_t	uint8_t

1.11 Parameter Set

Element domain storage. Since $p = 127 \leq 255$, all four levels store field elements in `uint8_t` (single byte); restricted exponents in $\mathbb{F}_z$ use 3 bits.

1.12 Data Size

Accurate calculation formulas.

$$\begin{aligned}
|\text{pk}| &= 2\lambda/8 (\text{Seed_pk}) + \lceil r \cdot 7/8 \rceil (\text{pack}(s)) \text{ bytes,} \\
|\text{sk}| &= 2\lambda/8 \text{ bytes (a single Seed_sk),} \\
|\text{sig}| &= \text{Salt} + 2 \cdot \text{Hash} + w \cdot \text{Seed} + w \cdot \text{Hash} + (t - w) \cdot (\lceil n \cdot 7/8 \rceil + \lceil n \cdot 3/8 \rceil + \text{Hash}).
\end{aligned}$$

data type	Qingluan-128	Qingluan-256	Qingluan-384	Qingluan-512
Public key $ \text{pk} $	77 B	153 B	227 B	302 B
Private key $ \text{sk} $ (single seed)	32 B	64 B	96 B	128 B
Signature $ \text{sig} $	18,720 B	74,248 B	164,940 B	292,816 B

Public key detail.

$$\text{Qingluan-128: } 32 + \lceil 51 \cdot 7/8 \rceil = 32 + 45 = 77 \text{ B.}$$

$$\text{Qingluan-256: } 64 + \lceil 101 \cdot 7/8 \rceil = 64 + 89 = 153 \text{ B.}$$

$$\text{Qingluan-384: } 96 + \lceil 149 \cdot 7/8 \rceil = 96 + 131 = 227 \text{ B;}$$

$$\text{Qingluan-512: } 128 + \lceil 198 \cdot 7/8 \rceil = 128 + 174 = 302 \text{ B.}$$

Signature detail ($y_{\text{bytes}} = \lceil n \cdot 7/8 \rceil$, $v_{\text{bytes}} = \lceil n \cdot 3/8 \rceil$, type-0 rounds = $t - w$):

$$\text{Qingluan-128: } 32 + 2 \cdot 32 + 212 \cdot 16 + 212 \cdot 32 + 44 \cdot (112 + 48 + 32) = 18,720 \text{ B.}$$

$$\text{Qingluan-256: } 64 + 2 \cdot 64 + 424 \cdot 32 + 424 \cdot 64 + 88 \cdot (220 + 95 + 64) = 74,248 \text{ B.}$$

$$\text{Qingluan-384: } \dots = 164,940 \text{ B;}$$

$$\text{Qingluan-512: } 128 + 2 \cdot 128 + 842 \cdot 64 + 842 \cdot 128 + 176 \cdot (430 + 185 + 128) = 292,816 \text{ B.}$$

2 Design Principles

2.1 The Mathematical Hard Problem on Which the Algorithm Is Based

The relationship between R-SDP and SDP. R-SDP restricts every coordinate to the multiplicative subgroup $E = \langle g \rangle$ (the constraint $e_i^z = 1$), rather than bounding a Hamming weight. Compared with SDP this yields:

- The problem stays hard: a solver must match the restricted structure (a z -th root of unity) on all n coordinates simultaneously.
- The restricted conditions can be efficiently verified within the MPC protocol.

Anti-quantum property analysis. No known efficient quantum algorithm exists for syndrome decoding. While Grover search offers a square-root speedup, its practical effectiveness is limited due to the inherent complexity of information-set decoding algorithms. The parameter selection for Qingluan has taken into account the impact of quantum acceleration.

2.2 Basis for Parameter Selection

2.2.1 Selection of the base field $\mathbb{F}_p$ and restricted subgroup E

- $p = 127$ is FIXED for all four levels: a 7-bit prime; one byte per element; rejection mask 0x7F with acceptance $127/128 > 99\%$.
- $z = 7$ and $g = 2$ give the restricted subgroup $E = \langle 2 \rangle = \{1, 2, 4, 8, 16, 32, 64\}$; restricted-ness is the algebraic constraint $e_i^7 = 1$.
- Unlike v1, the field does NOT grow with the level: security scales only through (n, k, t, w) , never through p , z , or g .

A prime field is chosen instead of power extensions of 2 because the modulo reduction operation over prime fields is simpler, and it avoids potential algebraic-structure attacks that may arise in extension fields.

2.2.2 Selection of code length n and dimension k

- Code rate $k/n \approx 0.596$ (anchored to CROSS round-2 Table 5); (n, k) reuse the CROSS R-SDP cat-1 / cat-5 analysed codes for 128 / 256 and are linear extrapolations (with redundancy) for 384 / 512.
- n is small (e.g. 127) because the hardness comes from the restriction $z = 7$, not from a huge code (contrast v1's $n = 2347$).

2.2.3 Selection of the round count t and fixed weight w

- (t, w) are chosen so the forgery cost (CROSS Prop 6 / Prop 7, the Kales–Zaverucha attack optimized for fixed weight) meets the classical target; the per-round soundness error is $\approx p/2(p-1) \approx 1/2$.
- Qingluan does NOT claim the CROSS ‘+5’ instruction-count credit, leaving ≈ 5 bits of forgery redundancy at every level.

2.2.4 The restricted subgroup E (fixed)

- E is the order-7 subgroup $\langle 2 \rangle$ of $\mathbb{F}_{127}^*$; its order $|E| = z = 7$ is fixed for all four levels.
- Exponents $\eta \in \mathbb{F}_z$ are sampled with a 3-bit rejection mask (accept if < 7); $e = g^\eta$ is obtained by a constant-time table lookup.

2.2.5 Why there is no party count N or seed tree

- v2 follows the CROSS 5-pass Σ -protocol: there is NO N -party MPC and NO GGM seed tree; soundness comes entirely from t fixed-weight parallel rounds.
- Each round contributes ≈ 1 bit (per-round soundness $\approx 1/2$), so $t \approx 2\lambda$ rounds are required (e.g. $t = 256$ for $\lambda = 128$).

2.2.6 Round count t and the fixed-weight challenge

- $\text{chall2} \in B(t, w)$ is a fixed-weight binary string of weight w : type-1 rounds reveal a seed, type-0 rounds reveal (y, v) .
- Qingluan-128: $t = 256$, $w = 212$ (raw forgery 128.8 bit).
- Qingluan-256: $t = 512$, $w = 424$ (raw forgery 257.2 bit).
- Qingluan-384: $t = 763$, $w = 631$ (384.3 bit); Qingluan-512: $t = 1018$, $w = 842$ (512.1 bit).
- Additional redundancy is employed to mitigate scenarios such as multi-target attacks.

2.3 Failure Probability Analysis

Qingluan signing draws a fresh per-signature **Salt** and root **Seed**; rejection sampling appears only when drawing uniform $\mathbb{F}_p / \mathbb{F}_z$ elements from the XOF and never rejects a whole signature. Therefore:

- The signature failure probability is 0 — a valid private key can always generate a valid signature for any message.
- The verification process is also well-defined: legitimate signatures are always accepted; invalid signatures are mistakenly accepted with a negligible probability (see the knowledge-reliability bound in the security statement for details).

- Rejection sampling only uniformizes $\mathbb{F}_p$ (7-bit mask, $> 99\%$) and $\mathbb{F}_z$ (3-bit mask, accept 7/8) on the XOF stream, with no impact on correctness.

Since no signature failure occurs, no re-signing mechanism is needed. Forgery success is governed by the CROSS fixed-weight forgery bound (Prop 6 / Prop 7) together with R-SDP hardness — not by a $(1/N)^\tau$ term — as detailed in the Security Analysis section.

2.4 Other Considerations

2.4.1 Selection of National Cryptographic Algorithms

Reasons for selecting the SM3 hash algorithm:

- The commercial cryptography standard approved by the National Cryptography Administration (GB/T 32905-2016).
- 256-bit output; Qingluan relies on SM3 preimage / second-preimage / TCR, widened by multi-pipe to 2λ bits — NOT on collision resistance beyond $\sim 2^{128}$ (Joux).
- Multi-pipe concatenation widens preimage / TCR to 256/512/768/1024 bits (it does not raise collision resistance), which is why every collision-relying use is salted.
- When used in conjunction with SM3-DRBG (GM/T 0005), it establishes a comprehensive national cryptographic random number generation system.

2.4.2 Constant-Time Implementation Design

The reference implementation of Qingluan employs a comprehensive constant-time design to defend against timing side-channel attacks.

Finite field arithmetic.

- All $\mathbb{F}_p$ operations ($p = 127$) use Barrett reduction instead of the modulo operator, avoiding data-dependent hardware division.
- The Barrett constant for multiplication: $M = \lfloor 2^K/q \rfloor$, where K is selected such that the estimation error ≤ 1 .
- Because $p = 127$ is fixed, a single Barrett constant $M = 129$, $K = 14$ serves all four levels.
- The Barrett accumulator constant (for inner product): $M = \lfloor 2^{32}/q \rfloor$, $K = 32$.
- Use an addition/subtraction condition mask instead of branching.

Restricted-set and exponent arithmetic.

- `restr_val` / `restr_exp_add` / `restr_exp_sub` use constant-time table lookups and masks over E (`restr.c`).
- g^η is computed by a branch-free table lookup over $E = \{1, 2, 4, 8, 16, 32, 64\}$.
- $v = \eta - \eta' \pmod{z}$ is computed without any secret-dependent branch.

Other constant-time measures.

- The comparison during signature verification employs `ct_memcmp` (a constant-time comparison based on XOR accumulation).
- Condition swap uses `ct_cswap` (mask-based branch-free swap).
- Use `secure_zero` for sensitive-data removal (resistant to compiler dead-store elimination).

2.4.3 Randomized signing with a deterministic KAT mode

Signing is randomized by a fresh **Salt** and root **Seed**; for KAT reproducibility a DRBG seed may be injected so the whole flow replays deterministically. The rationale:

- Simplify implementation and eliminate signature-failure paths.
- Easy testing: by fixing the DRBG seed, the entire signing process can be fully reproduced, facilitating KAT testing.
- The per-signature **Salt** doubles as the eTCR randomizer of the message digest (see Security §3); the DRBG (GM/T 0005) supplies high-entropy **Salt** and **Seed** from the OS CSPRNG.

2.4.4 Matrix-Structure Trade-offs

- $H = [V \mid I_{n-k}]$ is uniform random (NOT quasi-cyclic); only **Seed_pk** plus $\text{pack}(s)$ is stored, so the public key is still tiny (77–302 B).
- Because H is fully random, there is no quasi-cyclic algebraic structure for an attacker to exploit; hardness rests purely on R-SDP over the restricted subgroup E .

3 Safety Statement and Analysis

This section presents the security claims of the Qingluan digital signature scheme from three angles: theoretical security, concrete security, and other features. Under the Random Oracle Model (ROM) — and, conservatively, the Quantum ROM (QROM) — Qingluan meets EUF-CMA security based on the hardness of the Restricted Syndrome Decoding Problem (R-SDP) over the subgroup $E = \langle 2 \rangle$. The four parameter sets Qingluan-128/256/384/512 provide classical binding security of $\geq 129/257/384/512$ bits and quantum binding of $64/\geq 129/\geq 192/\geq 256$ bits (the 128 level meets its 80-bit quantum floor as NIST category 1; see the note below).

3.1 Theoretical Safety

3.1.1 Safety Model

Qingluan is argued EUF-CMA-secure in the ROM. Threat model (per algorithm requirements §2): the attacker may issue $\leq 2^{64}$ signing queries and $\leq 2^{80}$ hash (random-oracle) queries, under both classical and quantum computation (Grover square-root speedup applied conservatively). The model:

- The challenger runs $\text{KeyGen}(1^\lambda)$ to generate a key pair (pk, sk) and passes **pk** to attacker $\mathcal{A}$.
- Attacker $\mathcal{A}$ can adaptively initiate up to Q_s signature queries to the signature oracle, with each query message m_i yielding the corresponding signature σ_i .
- Attacker $\mathcal{A}$ can adaptively initiate up to Q_h queries against the hash oracle (SM3 and multi-pipe Hash).
- The final output $\mathcal{A} \rightarrow (m^*, \sigma^*)$ is considered successful if m^* has not been queried and σ^* is a valid signature for m^* .

The EUF-CMA advantage of the algorithm is denoted $\text{Adv}_{\text{Qingluan}}^{\text{EUF-CMA}}(\mathcal{A}) = \Pr[\mathcal{A} \text{ succeeds}]$.

3.1.2 Safety Theorem

Theorem 1 (EUF-CMA Security under ROM). *Under the random oracle model, if the R-SDP problem is (T, ε_{RSDP}) -hard, then the Qingluan signature scheme satisfies EUF-CMA security. Specifically, for any attacker $\mathcal{A}$ that initiates at most Q_s signature queries and Q_h hash queries within runtime T , there exists a solver $\mathcal{B}$ for the R-SDP problem (with runtime $\approx T$) such that*

$$\begin{aligned} \text{Adv}_{\text{Qingluan}}^{\text{EUF-CMA}}(\mathcal{A}) \leq & \text{Adv}^{\text{R-SDP}}(\mathcal{B}) + \underbrace{(\text{forgery bound from CROSS Prop 6 / Prop 7})}_{\text{fixed-weight grinding}} \\ & + \underbrace{(\text{multi-target eTCR term})}_{\text{salted message digest}}. \end{aligned}$$

There is NO $(1/N)^\tau$ term — v2 has no N -party MPC.

The meaning of each item:

- $\text{Adv}^{\text{R-SDP}}(\mathcal{B})$: the advantage of solving a restricted instance over $E = \langle 2 \rangle$, governed by (n, k) and the restriction $z = 7$.
- Forgery term: the cost of grinding the fixed-weight Fiat–Shamir challenge (Prop 6 / Prop 7 = the Kales–Zaverucha attack), at least 128.8/257.2/384.3/512.1 bits raw for the four levels.
- Message-binding term: multi-target eTCR / second-preimage of the salted `digest_Msg`, $\approx 2^{2\lambda-64}$; this is NOT a birthday/collision term (see the hash-reduction discussion).

Proof approach (two-step reduction).

- *Special soundness*: from two accepting transcripts that share $(\text{cmt0}, \text{cmt1}, \text{chall1})$ but differ in `chall2`, one extracts a restricted $e = v \star e' \in E^n$ with $eH^\top = s$. Knowledge of such e is R-SDP-hard.
- *Fiat–Shamir in the ROM* turns soundness into unforgeability: a cheating prover must grind the challenge at cost $\text{forgery}_{\text{raw}}$, already accounting for the 2^{80} hash-query bound.

3.1.3 Security under the Quantum Computing Model

Under the Quantum Random Oracle Model (QROM), the security analysis of the Fiat–Shamir transform is more complex; however, universal QROM security results such as the Unruh transform or the Don–Fehr–Majenz–Schaffner (DFMS) approach have been established for interactive protocols with up to five rounds. After applying the Fiat–Shamir transform to Qingluan’s MPCitH protocol, these results can be utilized to achieve EUF-CMA security under QROM, with a constant-factor reduction in tightness compared to ROM (the specific tightness bound depends on the security theorem employed).

Qingluan covers the QROM reduction loss conservatively by setting `quantum_bits = classical/2` (full Grover on both forgery search and ISD); the multi-pipe Hash gives quantum preimage 2^λ , covering the quantum one-wayness needed for challenge derivation.

3.2 Concrete Safety Measures

For the declared 128/256/384/512-bit levels, the following gives attack-cost estimates (source: `rsdp_estimator.py`, which reproduces CROSS round-2 Table 5 / Table 7).

3.2.1 Classical Computational Attack

Information Set Decoding (ISD) Attack.

Cost model ($\log_2$)	Qingluan-128	Qingluan-256	Qingluan-384	Qingluan-512
Key recovery (ISD w/ representations, classical)	2^{143}	2^{274}	2^{400}	2^{528}
Fiat–Shamir forgery (CROSS Prop 6/7, classical)	$2^{128.8}$	$2^{257.2}$	$2^{384.3}$	$2^{512.1}$
Binding security = min (classical)	2^{129}	2^{257}	2^{384}	2^{512}

ISD (with representations and error-set shifting) is the best known key-recovery attack against R-SDP; Qingluan anchors its cost to CROSS round-2 Table 5. Each level’s classical binding security $\geq$ its target ($129/257/384/512 \geq 128/256/384/512$), with the 384/512 codes carrying ~ 15 bits of key-recovery margin (linear extrapolation, to be re-checked with the official CROSS estimator before standardization).

3.2.2 Algebraic Attack

- Because $H = [V \mid I_{n-k}]$ is uniform random, there is no quasi-cyclic (block-circulant) structure for an attacker to exploit algebraically.
- Hardness reduces purely to R-SDP over the subgroup E ; the small n is compensated by the restriction $z = 7$, which is the actual source of hardness.

3.2.3 Fiat–Shamir Forgery (Fixed-Weight Grinding)

Bypassing R-SDP means grinding the fixed-weight Fiat–Shamir challenge; its cost is the CROSS Prop 6 / Prop 7 forgery bound (Kales–Zaverucha, optimized for fixed weight):

- Qingluan-128: $\geq 2^{128.8}$ forgery attempts (raw, no +5 instruction-count credit).
- Qingluan-256: $\geq 2^{257.2}$.
- Qingluan-384: $\geq 2^{384.3}$; Qingluan-512: $\geq 2^{512.1}$.

Even at 2^{80} hash queries the per-level forgery cost stays above the classical target — the query bound is already folded into the Prop 7 (Kales–Zaverucha) analysis.

3.2.4 Quantum Computing Attacks

Grover-accelerated ISD. Qingluan applies a conservative full-Grover square root (quantum = classical/2) to BOTH key recovery and forgery:

Cost model ($\log_2$, quantum = classical/2)	Qingluan-128	Qingluan-256	Qingluan-384	Qingluan-512
Key recovery (quantum)	2^{72}	2^{137}	2^{200}	2^{264}
Fiat–Shamir forgery (quantum)	$2^{64.4}$	$2^{128.6}$	$2^{192.2}$	$2^{256.1}$
Binding security = min (quantum)	2^{64}	2^{129}	2^{192}	2^{256}

- Qingluan-128: quantum binding 64 ($= 128/2$ full Grover); the 80-bit floor is met as NIST category 1 (AES-128 under bounded-depth Grover), not as a literal 2^{80} .
- Qingluan-256: quantum binding $\geq 129 \geq 128$.
- Qingluan-384: quantum binding $\geq 192 \geq 192$; Qingluan-512: $\geq 256 \geq 256$.

3.2.5 Grover-accelerated signature forgery

Quantum forgery search gains at most a square-root speedup over the classical fixed-weight grinding bound, giving the quantum forgery row above (64.4/128.6/192.2/256.1).

- Qingluan-128: quantum forgery $2^{64.4}$ (met as NIST category 1).
- Qingluan-256: quantum forgery $2^{128.6} \geq 128$.
- Qingluan-384: $2^{192.2} \geq 192$; Qingluan-512: $2^{256.1} \geq 256$.

parameter set	Classical target	Actual classical binding	Quantum target	Actual quantum binding
Qingluan-128	128 bit	≥ 129 bit	≥ 80 bit	64 bit (NIST cat-1)
Qingluan-256	256 bit	≥ 257 bit	≥ 128 bit	≥ 129 bit
Qingluan-384	384 bit	≥ 384 bit	≥ 192 bit	≥ 192 bit
Qingluan-512	512 bit	≥ 512 bit	≥ 256 bit	≥ 256 bit

3.2.6 Summary of Concrete Safety Strengths

All four sets meet their classical targets with redundancy; quantum meets 256/384/512 outright, and 128 as NIST category 1.

3.3 Other Safety Features

3.3.1 Security Against Side-Channel Attacks

The Qingluan reference implementation employs a comprehensive constant-time design to defend against timing side-channel attacks.

- Finite field arithmetic: all $\mathbb{F}_q$ operations use Barrett reduction to replace modulo operators, eliminating secret correlations.
- Restricted error generation: $e = g^\eta$ is built by a constant-time table lookup over $E = \{1, 2, 4, 8, 16, 32, 64\}$; the access pattern is independent of the secret exponents η .
- Syndrome / round arithmetic: constant-time matrix-vector products with the public $H = [V \mid I]$; memory access depends only on public data.
- Security comparison: the `ct_memcmp` function (XOR accumulation) is used in signature verification for constant-time comparisons.
- Condition swap: use `ct_cswap` (mask-based branch-free swap).
- Sensitive-data removal: the `secure_zero` function resists compiler dead-store elimination.
- DRBG state: thread-local storage, preventing multithreading contention and state leakage.

The above measures can withstand mainstream timing side-channel attacks. For physical side-channel threats such as electromagnetic and power analysis, additional hardware-level protections (e.g. shielding or masking) are required; these fall outside the scope of algorithmic specifications.

3.3.2 Forward Safety

The base design does not provide forward security: the private key is a single seed `Seed_sk`, and an attacker who obtains it at time T can forge past messages. Forward security can be added as follows:

- Regular key evolution: periodically re-derive `Seed_sk` and securely erase the previous key material.
- By integrating hash chains or tree structures, secret evolution can be achieved while maintaining the stability of the public key.
- The above extension falls under application-layer protocol design and does not modify the core signature algorithm.

The per-signature `Salt`, root `Seed`, and round seeds are wiped by `secure_zero` after signing, so a later sk compromise cannot reveal the internal state of past signatures.

3.3.3 Security Against Multi-Key Attacks

In multi-user / multi-key scenarios, an attacker obtains K independent public keys and attempts to forge any one of them. The analysis:

- Key independence: each key pair uses an independent DRBG state (seeded by a CSPRNG), with no structural correlation.
- Security degradation: in multi-user scenarios, security is inherently degraded by an $O(K)$ factor, meaning the attacker's advantage in forging any of the K keys is $\leq K \times \text{Adv}^{\text{EUF-CMA}}(\mathcal{A})$.
- Round redundancy ($t \approx 2\lambda$ rounds) absorbs reasonable multi-user scale ($K \leq 2^\lambda$).
- Each user has an independent `Seed_pk` (hence an independent uniform $H = [V \mid I]$) and `Seed_sk`, so there is no algebraic correlation across users.

3.3.4 Temporary Key Reuse

The primary random seed (`master_seed`) used in the Qingluan signature process is generated by SM3-DRBG and derived independently for each signature. The implications of reusing temporary random numbers:

- Each signature draws a fresh `master_seed` and a fresh 2λ -bit `Salt` from SM3-DRBG (`sign.c`), so the t round seeds $\text{Seed}[i] = \text{CSPRNG}(\text{master_seed} \parallel \text{Salt} \parallel i)$ never repeat across signatures — even of the same message — and the per-round commitment randomness (η', u') is never reused.
- If the entire DRBG is reset (generating the same `master_seed`) and signing the same message — with definitive and consistent signature results — no additional information is disclosed.
- If the DRBG is reset across two different messages, v2's round seeds — derived from `master_seed` and `Salt`, not from the message digest μ as in v1 — would repeat while the Fiat-Shamir challenges differ; v2 therefore keeps every round's randomness unique through fresh per-signature `master_seed` and `Salt` (drawn anew each call, then wiped by `secure_zero`), not through message-derived seeds.

In conclusion, Qingluan demonstrates strong robustness under DRBG temporary-state reuse. However, it is strongly recommended to strictly adhere to the DRBG reseeding rules specified in GM/T 0005 to maintain entropy-source quality.

3.3.5 Compliance of the GM/T 0028 Cryptographic Module

Implement a self-check interface compliant with the GM/T 0028 cryptographic-module standard:

- `crypto_sign_signtest()`: comprehensive self-test that returns pass/fail in one attempt.
- `crypto_sign_signtest_detailed()`: sub-report containing test results for SM3 KAT, DRBG functionality, KeyGen, Sign, and Verify.

Self-check items include:

- Known-answer test for the SM3 hash algorithm (comparison with the standard vector in Appendix A of GB/T 32905-2016).
- SM3-DRBG functional test (initialization, generation, reseeding path).
- KeyGen generates a valid key pair (in pk/sk format and specified length).
- Check the correctness of the Sign/Verify operation (self-validation loop).
- Test vectors `KAT_SIG_Qingluan-{128,256,384,512}.txt` (API_PKC framework). All four parameter sets reproduce the KeyGen→Sign→Verify loop (10 vectors each).

3.3.6 Summary of Safety Statements

Based on the above analysis, the Qingluan digital signature scheme declares the following security features:

- Under ROM, EUF-CMA security holds from R-SDP hardness, with the witness genuinely bound into the commitment so that v1's public-key universal forgery is structurally impossible (regression-gated by `test/test_forgery.c`).
- Under QROM (conservative full-Grover), the four sets achieve classical binding $\geq 129/257/384/512$ and quantum binding $64/\geq 129/\geq 192/\geq 256$ (128 as NIST cat-1), meeting the 128/256/384/512 and 80/128/192/256 targets.
- The reference implementation provides constant-time defense capabilities and resists timing side-channel attacks.
- In multi-user scenarios, security is naturally downgraded to an $O(K)$ factor, with parameter redundancy sufficient to handle typical scales.
- The DRBG state-reuse mechanism exhibits robustness.
- Provides a cryptographic-module self-check interface compliant with GM/T 0028.

4 Performance Evaluation

This section provides a performance analysis of the Qingluan algorithm at the design-principle level, along with reference performance test results on mainstream 64-bit PC processors, and compares its performance with existing standard signature algorithms.

4.1 Performance Analysis

4.1.1 Computational Complexity Analysis

The core of Qingluan is $\mathbb{F}_p$ ($p = 127$) matrix-vector products plus SM3 hashing — no Gaussian sampling, no floating point, no elliptic-curve or pairing arithmetic. The cost is dominated by the t parallel rounds (there is no N -party factor). The three main operations have the following complexity:

operation	Main operation	Asymptotic complexity
KeyGen	Expand V ; one syndrome $s = V \cdot e_A^\top + e_B$	$O(r \cdot k) \mathbb{F}_p$ mult.
Sign	t rounds: $u = v_E \star u'$, $s' = uH^\top$, $y = u' + \text{chall1} \cdot e'$	$O(t \cdot r \cdot n) \mathbb{F}_p$ mult + $O(t)$ hashes
Verify	Recompute the t rounds (cmt0 / cmt1 branches)	$O(t \cdot r \cdot n) \mathbb{F}_p$ mult + $O(t)$ hashes

4.1.2 Specific Computational Requirements

parameter set	$r \cdot n$	Sign $\mathbb{F}_p$ mult $\approx t \cdot r \cdot n$	Sign hashes $\approx 2t$
Qingluan-128	$51 \times 127 = 6,477$	$256 \times 6,477 \approx 1.66\text{M}$	512
Qingluan-256	$101 \times 251 = 25,351$	$512 \times 25,351 \approx 12.98\text{M}$	1,024
Qingluan-384	$149 \times 370 = 55,130$	$763 \times 55,130 \approx 42.06\text{M}$	1,526
Qingluan-512	$198 \times 491 = 97,218$	$1018 \times 97,218 \approx 98.97\text{M}$	2,036

Verify is the same order as Sign. KeyGen is a single V expansion plus one syndrome, far cheaper ($\approx 1/t$ of Sign).

4.1.3 Transmission and Storage Overhead Analysis

data type	Qingluan-128	Qingluan-256	Qingluan-384	Qingluan-512
Public key (PK)	77 B	153 B	227 B	302 B
Private key (SK)	32 B	64 B	96 B	128 B
Signature (SN)	18,720 B	74,248 B	164,940 B	292,816 B

Characteristics:

- Public key 77–302 B — seed-compressed ($\text{Seed_pk} + \text{pack}(s)$); far smaller than most PQC public keys and only marginally larger than RSA/ECDSA.
- The private key is extremely small — only 32 to 128 bytes in seed format, facilitating key custody and distribution.
- The large signature (18–286 KB) is the inherent MPCitH cost: w revealed seeds + w commitments + $(t - w)$ ($y, v, \text{cmt1}$) response triples.

4.1.4 Comparative Analysis with Existing Standard Algorithms

Compared with classic signature algorithms at the 128-bit classical security level:

algorithm	type	public key	signature	Anti-quantum
Qingluan-128	coding	77 B	18.3 KB	yes
RSA-3072	integer factorization	384 B	384 B	no
ECDSA P-256	elliptic-curve discrete log	64 B	64 B	no
SM2	elliptic-curve discrete log	64 B	64 B	no

Classic algorithms hold advantages in computational complexity, yet none possess resistance against quantum attacks. Shor’s quantum algorithm can break RSA/ECDSA/SM2 in polynomial time — a fundamental reason for the existence of Qingluan.

Comparison with post-quantum signature schemes (128-bit classical security). Comparative conclusions at the design-principle level:

- Public key (77 B): smaller than Dilithium2 and Falcon-512, comparable to CROSS — among the smallest PQC public keys.
- Signature (18–286 KB): the inherent MPCitH cost, comparable to CROSS and larger than lattice signatures.
- Computational efficiency: the core relies on linear algebra and hash operations, with no floating-point operations or complex algebraic structures, making it suitable for embedded hardware implementation.
- Diversity of security assumptions: leveraging coding cryptography, Qingluan provides distinct hard-problem choices for PQC algorithm suites beyond lattice/hash schemes, enhancing diversified cryptographic defense strategies.
- National-cryptography integration: the only post-quantum signature scheme that employs SM3 as its underlying primitive, meeting domestic compliance requirements.

4.2 Performance Testing

4.2.1 Detailed Platform Configuration

Implementation method.

- Programming language: ISO C11.
- Compiler: GCC; optimization level: 2; tool: GNU Make.
- Dependent library: only the C standard library.

Software implementation platform.

- Processor: x86-64 (64-bit general-purpose processor).
- Operating system: Windows 11 Pro (kernel version 10.0.26200).
- Instruction set: x86-64 basic instruction set.
- Crypto library: no external dependencies; all crypto primitives are implemented locally.
- Random entropy source: Windows BCryptGenRandom.

Code-path implementation.

- Reference implementation: `Implementations/Reference_Implementation/`.
- Per-instance KAT generator: each algorithm instance carries its own `api_pkc/` folder whose `hash_adapter.c` rebuilds the same multi-pipe SM3 construction as `src/hash.c`, so its KAT vectors match the reference implementation byte-for-byte.
- All source code complies with C11 and is cross-platform compilable.

4.2.2 Software Implementation Performance Test Results

On the platform above, each instance’s `api_pkc/` KAT generator produces 10 KAT vectors, each running the full KeyGen→Sign→Verify loop. Reference single-signature timing (MinGW GCC, -02):

parameter set	Single Sign (reference)	Verify ($\approx$ Sign)	#KAT vectors
Qingluan-128	~ 77 ms	~ 77 ms	10
Qingluan-256	~ 0.23 s	~ 0.23 s	10
Qingluan-384	~ 0.42 s	~ 0.42 s	10
Qingluan-512	~ 0.87 s	~ 0.87 s	10

v2 single-signature time is milliseconds to under one second — versus v1’s 2.7 s–139 s — because there is no N -party loop, only t rounds. Verify $\approx$ Sign; KeyGen $\ll$ Sign:

parameter set	KeyGen	Sign	Verify
Qingluan-128	< 1 ms	~ 77 ms	~ 77 ms
Qingluan-256	< 3 ms	~ 0.23 s	~ 0.23 s
Qingluan-384	< 6 ms	~ 0.42 s	~ 0.42 s
Qingluan-512	< 12 ms	~ 0.87 s	~ 0.87 s

Signing time grows $\approx 3\times$ per level step, matching the $t \cdot r \cdot n$ cost growth; an optimized implementation (Barrett, loop unrolling, multi-accumulator matmul) is Part-2 scope and targets further speedups.

data type	Qingluan-128	Qingluan-256	Qingluan-384	Qingluan-512
public key (PK)	77 B	153 B	227 B	302 B
private key (SK, seed format)	32 B	64 B	96 B	128 B
signature (SN)	18,720 B	74,248 B	164,940 B	292,816 B

4.2.3 Transmission and Storage Overhead

Test-vector file size:

KAT file	file size (approx.)	remarks
KAT_SIG_Qingluan-128.txt	≈ 0.4 MB	verified round-trip
KAT_SIG_Qingluan-256.txt	≈ 1.5 MB	verified round-trip
KAT_SIG_Qingluan-384.txt	≈ 3.3 MB	verified round-trip
KAT_SIG_Qingluan-512.txt	≈ 5.9 MB	verified round-trip

The signature scheme is non-interactive, requiring no key-exchange messages or multiple rounds of interaction, thus eliminating additional round counts or communication overhead.

4.2.4 Software Implementation Resource Consumption (Optional)

Estimated working memory (Qingluan-128 example, bytes):

- Matrix V ($r \times k = 51 \times 76$ elements): $51 \times 76 \times 1 \text{ B} \approx 3.9 \text{ KB}$.
- Error / exponent vectors ($n = 127$ elements each): η and $e \approx 127 \text{ B}$ each.
- Per-round vectors (u', v, y over $\mathbb{F}_p^n / \mathbb{F}_z^n$): a few hundred bytes; rounds are processed sequentially.
- Round seeds ($t \times \text{SEED}$): $256 \times 16 = 4,096 \text{ B}$ (no $2N$ -node tree).
- Commitment buffers ($t \times \text{HASH}$, for cmt0 and cmt1): $256 \times 32 = 8,192 \text{ B}$ each.
- Signature buffer: $\approx 18 \text{ KB}$ (Qingluan-128).
- Peak working memory: tens of KB (128) up to a few MB (512) — far below v1, which had an $r \times n \approx$ millions-entry matrix and an N -party share array.

Memory is now dominated by the small V matrix and the t round buffers; embedded platforms can stream the rounds.

4.2.5 Performance Comparison with Existing Standard Algorithms

On the same x86-64 / Windows 11 platform, using OpenSSL as the classical-algorithm baseline (for approximate performance reference and order-of-magnitude comparison):

algorithm	KeyGen	Sign	Verify	public key	signature
RSA-3072	$\sim 100 \text{ ms}$	$\sim 3 \text{ ms}$	$\sim 0.1 \text{ ms}$	384 B	384 B
ECDSA P-256	$\sim 0.1 \text{ ms}$	$\sim 0.1 \text{ ms}$	$\sim 0.3 \text{ ms}$	64 B	64 B
SM2	$\sim 0.1 \text{ ms}$	$\sim 0.1 \text{ ms}$	$\sim 0.3 \text{ ms}$	64 B	64 B
Dilithium2	$\sim 0.1 \text{ ms}$	$\sim 0.5 \text{ ms}$	$\sim 0.1 \text{ ms}$	1.3 KB	2.4 KB
Qingluan-128 (reference)	$< 1 \text{ ms}$	$\sim 77 \text{ ms}$	$\sim 77 \text{ ms}$	77 B	18.3 KB

Qingluan's latency (tens of ms to under a second) is higher than classical and lattice signatures but now practical; it comes from the t parallel rounds (no N -party factor). FPGA/ASIC can accelerate 10–100 $\times$ in later evaluation rounds.

5 Feature Statement

5.1 Innovation of the Algorithm

- **R-SDP-based MPC-in-the-Head signature:** Qingluan instantiates the CROSS-RSDP protocol, proving knowledge of a restricted error $e \in E^n$ ($E = \langle 2 \rangle$) with $eH^\top = s$ through a 5-pass Σ -protocol. A uniform-random $H = [V \mid I]$ keeps the public key to a single seed plus the syndrome.
- **Fixed-weight parallel rounds:** soundness comes from t rounds with a weight- w binary challenge; the signature reveals w seeds, w commitments, and $(t - w)$ $(y, v, \text{cmt1})$ response triples (the direct fast-style form — seed-tree / Merkle compression is Part-2 scope and changes only size).
- **Multi-pipe SM3 hash:** concatenating HASH_PIPES domain-separated SM3 pipes widens preimage / second-preimage / TCR resistance to 2λ bits (256/512/768/1024). It does NOT raise plain collision resistance beyond $\sim 2^{128}$ (Joux), so every collision-relying use is salted (eTCR).

- **Deep integration of national-cryptography primitives with PQC signatures:** this solution uniformly incorporates the SM3 hash algorithm, SM3-DRBG (GM/T 0005), and the MPCitH signature paradigm, representing one of the earliest full-chain integration solutions combining national cryptography and PQC.
- **Witness-binding fix:** the secret-dependent $v = \eta - \eta'$ enters the commitment `cmt0`, and the verifier checks $v \in \mathbb{F}_z^n$ (restricted-ness) plus the linear relation — so `v1`'s universal forgery from the public key alone is structurally impossible. Signing still has zero failure probability.

5.2 Simplicity of the Algorithm

- The algorithm core involves only $\mathbb{F}_p$ ($p = 127$) linear algebra (matrix-vector products) and the SM3 hash function.
- It does not involve Gaussian sampling, rejection sampling (used only for element sampling in uniform domains with an acceptance rate $> 98\%$, without affecting signature accuracy), floating-point operations, or elliptic-curve operations.
- Signing is randomized by a fresh per-signature `Salt` and root `Seed`, yet has zero failure probability and needs no re-signing loop.
- The reference implementation source code is concise, with its core files (`keygen.c` / `sign.c` / `verify.c` / `mpc.c`) each limited to a few hundred lines, facilitating auditing and third-party implementation.
- Minimally designed cryptographic primitives: SM3, multi-pipe SM3, SM3-DRBG, and SM3-based XOF, all relying solely on a single underlying hash function.
- No external crypto-library dependencies; all components are self-implemented, eliminating dependency-chain risks.

5.3 Flexibility of the Algorithm

- Four preset parameter sets (128/256/384/512) cover applications from general use to defense-grade security.
- The signature-size / speed trade-off is tuned via (t, w) : the estimator's `-search` mode scans the (t, w) curve to derive fast / small variants for the same code (n, k) .
- Unified parameter framework: the four levels share one code base and algorithm structure; only (n, k, t, w) differ, while the base algebra ($p = 127$, $z = 7$, $g = 2$) stays fixed via `PARAM_*` macros — no need to maintain multiple code versions.
- Optional intermediate levels (e.g. 192-bit) are obtained by interpolating (n, k, t, w) without changing the algorithm structure.
- The message length is unlimited and supports input of any length.

5.4 Algorithm Compatibility

- The algorithm structure is compatible with the standard digital-signature interface.
- C language compatible with NIST PQC standards. API: `crypto_sign_keypair`, `crypto_sign`, `crypto_sign_open`.
- Supports two interfaces: the attached mode (`crypto_sign` + `crypto_sign_open`, combining signature and message) and the separated mode (`crypto_sign_signature` + `crypto_sign_verify`, using only the signature).

- Provides adapter interfaces compatible with the NGCC API_PKC framework (`sig_keygen` / `sig_sign` / `sig_verify`), validated by the API_PKC KAT testing framework.
- It seamlessly replaces existing signature schemes — the caller only needs to adjust the key or signature byte-count constants.
- Compatible with the standard X.509 certificate system and PKI infrastructure (where public keys / signatures are transmitted as byte strings, requiring no special encapsulation).
- Provides the GM/T 0028 cryptographic-module self-test interfaces (`crypto_sign_selftest`, `crypto_sign_selftest_detailed`) to facilitate integration with compliant cryptographic modules.

5.5 Scalability of the Algorithm

- The structure extends naturally to higher levels by increasing (n, k, t, w) ; it already covers 512-bit classical security.
- The MPCitH framework can be extended to multi-party threshold signatures — where multiple key holders collaboratively generate a signature without disclosing their individual keys. This extension aligns perfectly with MPCitH's inherent capabilities.
- It can be integrated with similar key-encapsulation (KEM) schemes to form a complete algorithm suite, featuring shared underlying SM3-DRBG and multi-pipe hash implementations.
- The multi-pipe SM3 architecture scales the hash width by adding pipes; four pipes already provide 1024-bit preimage / TCR width (the 512 level).
- The fixed-weight rounds can be batched across multiple messages to amortize the per-signature cost.
- The reserved hardware-acceleration interface — the finite-field arithmetic module (`fq_arith`) and hash module — can be independently replaced with hardware-accelerated versions without affecting upper-layer protocols.

5.6 Cross-Platform Implementation Performance Advantages

- The core operations involve addition and multiplication over finite fields, making them compatible with various processor architectures (e.g. x86, ARM, RISC-V).
- The Barrett reduction technique replaces modulo operations, eliminating reliance on hardware dividers — allowing efficient implementation even on embedded processors lacking division instructions.
- The t independent rounds and the $\mathbb{F}_p$ matrix-vector products map directly to SIMD instruction sets such as AVX2 / NEON / RVV.
- No floating-point operations, ideal for embedded systems and hardware implementation — FPGA/ASIC-friendly with pure-integer data paths.
- Direct implementation with constant time complexity enhances resistance to side-channel attacks — techniques such as linear scanning, Barrett reduction, and `ct_memcmp` require no specialized architectural support.
- The DRBG uses thread-local storage, supporting concurrent signing/verification across multiple threads — enabling linear throughput scaling on multi-core servers.

- Cross-platform support: both Windows and POSIX entropy sources are supported; compilation requires only standard C11.
- National-cryptography compliance advantage: in scenarios requiring national-cryptography algorithms (government, enterprise, finance, defense), Qingluan can directly replace PQC schemes based on non-national-cryptography hash functions without additional compliance reviews.
- Low-power advantage: no floating-point unit is required, no reliance on large memory pages, and signature computation can be performed with lower power consumption on embedded secure elements (SE) or IoT devices for KeyGen/Verify.